

XL

XL-AGENT

产品报告 · PRODUCT REPORT

XL-Agent 产品报告

迅雷 AI 可信资源供给层

产品设计思路

用户需求分析

功能架构

AI 能力说明

落地可行性

商业化思考

2026 · 08

可信 · 可控 · 可审计 · 可复现

XL-Agent / 迅雷 AI

目录

从产品洞察到可信供给架构，再到落地与商业化路径。

01 产品设计思路

02 用户需求分析

03 功能架构

04 AI 能力说明

05 落地可行性

06 商业化思考

07 一句话总结

报告围绕 XL-Agent 的核心定位展开：用 AI 编排资源准备任务，以迅雷网络承载可信到达，并通过状态机与审计链守护执行边界。

1. 产品设计思路

1.1 核心洞察

过去二十年，迅雷解决的是"下载"；未来十年，用户要的不是链接，是"把一件事做完"。

AI Agent 时代带来一个被普遍忽略的断层：Claude Code、Codex 这类 Coding Agent 能写代码、跑命令，但它们**依赖环境、依赖、工具从哪来？谁来保证可信、可复现、快速到达？**

- Coding Agent 是"消费者"：默认信任公共索引的"最新"，不做供应链验证，不可审计。
- 迅雷拥有二十年积累的 P2SP 分发网络——**供给侧的物理资产，Agent 公司一家都没有。**

因此 XL-Agent 不做"又一个写代码的 Agent"，而是做**所有 Agent 上游的可信供给层**：用 AI 编排任务，用迅雷网络承载到达，用状态机守护边界，用审计建立信任。

1.2 设计哲学：默认只读，执行是特权

原则	实现
模型没有 Shell	Agent Loop 只授予只读能力信封；未知工具直接拒绝
写入必须双审批	下载/写盘/导出进入新 plan revision，用户单独确认，审批带 TTL 过期失效
生成与审计分离	主 Agent 生成工作区，Agent B 以最小权限 grant 只读复查
一切可追溯	SQLite 记录任务、revision、审批、下载断点、验证结果、操作事件
传输可插拔	下载适配器接口统一：受控 HTTPS（默认）↔ 迅雷 C++ SDK P2SP（已实现并实测）

1.3 关键取舍

- 不做任意命令执行 / 不自动安装**：这是安全基线，不是能力缺失。架构上已预留 `code_execution` 风险等级，路线图以"白名单参数的受控安装"方式开放，仍不放开任意 shell。
- 能力层认输，治理层碾压**：环境检查、写代码这类 Coding Agent 的主场不强争；可信供给、双审批、供应链验证、审计链是差异化价值。

2. 用户需求分析

2.1 目标用户分层

用户	典型诉求	对应能力
开发者	"把 DeepSeek 的 harness 项目拉到本地跑起来"	GitHub 检索 → 固定 commit → 分析 → 受控下载 → 交接
AI 工程师	"我要用 PyTorch, 看环境还缺什么"	只读环境分析 + 兼容性报告 + 资源准备
科研人员	"准备一个科研数据分析工作区"	科研 Domain Skill + 可信目录资源编排
企业/团队	"环境要一致、来源要可信、出了事能追溯"	供应链验证、双审批、审计、Agent B 复核

2.2 用户痛点 → 方案映射

痛点	现状 (Coding Agent / 手动)	XL-Agent 方案
装错版本 / 依赖漂移	装"最新", 昨天能跑今天不能跑	固定 commit + 锁文件 SHA512 绑定
供应链攻击	公共索引被投毒无法感知	可信目录 + Authenticode + 哈希双校验
不可控	Agent 乱跑命令、越权	状态机裁决 + 能力信封 + 双审批
不可审计	跑了什么无法追溯	SQLite 全量事件 + 审批链
大文件慢 / 断线	wget 挂了就重来	流式下载、断点续传、迅雷 P2SP 加速
交接混乱	拿到一堆文件不知道干嘛	交接包: manifest + README + AGENTS.md + 验证脚本

2.3 需求边界 (诚实声明)

- 当前 MVP 面向**开发者/供给层**, 普通用户"一键安装"走路线图 (受控静默安装)。
- "资源已准备"不等于"软件已安装"——安装决策权留给人与下游 Agent。

3. 功能架构

3.1 分层架构

```
Renderer (React + TS)  仅展示状态 / 派发白名单事件
preload (contextBridge)  类型化白名单 IPC
Electron Main (Orchestrator)
  AgentRuntimeHost → AgentRuntime → 状态机
  Tool Registry + Policy + SQLite + 下载/验证/导出
  LocalXunleiAdapter → NativeXunleiDownloadClient
    → xunlei-download-host.exe → dk.dll (P2SP)
Agent Core (src/features/agent-core)
  router / taskPlan / machine / agentLoop / policy
```

安全边界： `nodeIntegration:false` + `sandbox:true` , Renderer 无 Node/文件系统/SQLite 访问；模型、Policy、工具、持久化全部闭环在主进程。

3.2 状态机（机器约束）

```
intake → routing → task_planning → clarifying → planning
  → waiting_approval → downloading → verifying → exporting → handoff
    ↘ replanning / awaiting_failure_action / awaiting_export_retry ✓
```

- `transition(state, event)` 纯函数，非法事件原样返回；约 50 种事件、全程审计日志。
- 写入必经：新 plan revision → 用户确认 → 审批（TTL 过期失效）。

3.3 Agent Loop 内核（受控 ReAct）

- 只读能力信封（`maxRisk: "read_only"`）+ 预算约束（轮次/工具数/重复调用/墙钟时间）。
- 工具名、参数、返回 Zod 严格校验；并行只读；事件序列单调可审计；断点续跑。

3.4 供应链验证链

```
可信目录（版本化 + SHA256 指纹）
  → 来源白名单（HTTPS + allowedHosts）
  → 下载（流式/断点/迅雷 P2SP）
  → 大小 / SHA256 / Authenticcode 发布者 / 锁文件 SHA512 四重校验
  → 原子写入工作区交接包 → Agent B 只读复查
```

3.5 主要模块清单

模块	状态
任务路由（确定性 + LLM 语义分类）	✓
TaskPlan DAG（revision/审批/澄清/恢复）	✓
Agent Loop 内核	✓
受控下载（HTTP + 迅雷 P2SP 可插拔）	✓（P2SP 已实测）
供应链验证（哈希/签名/锁文件）	✓
工作区交接（原子写入 + Manifest + AGENTS.md）	✓
Agent B 只读复查（grant + TTL）	✓
SQLite 持久化与恢复	✓
GitHub 检索 / 固定 commit / 独立审批准布	✓
任务历史与审计 UI	✓
受控安装（静默安装器）	✖ 路线图

4. AI 能力说明

4.1 已实现

能力	说明
意图路由	确定性规则（关键词/链接/仓库名提取）+ 远程 LLM 语义分类双通道，unsupported 时才问模型
计划生成	TaskPlan 提议（目标/步骤/工具/风险/交付物），模型提议、人确认
受控推理循环	模型 → 工具 → 观察 → 下一步，预算与能力信封约束，防无限循环
项目兼容性分析	读取固定 commit 的 README/清单/锁文件，与本机环境只读对比，输出满足/缺失/待确认
本地规则模型	无需 API Key 即可演示完整流程（确定性规则实现）
远程模型	OpenAI-compatible / DeepSeek，失败熔断回退本地规则
Agent B 审计	独立只读复查 manifest，返回 revision/已准备项/允许与禁止动作

4.2 能力边界（设计声明）

- 模型无 Shell、不能提交任意下载 URL、不能绕过 Policy。
- 不自动安装、不执行仓库代码——安全基线，架构预留 `code_execution` 等级供受控开放。

5. 落地可行性

5.1 已完成并实测

- **迅雷 P2SP 真实下载链路已打通**： `xl_dl_init` → `login` → `create_p2sp_task` → 轮询 → `completed`，实测下载 Python 3.12.10 (25.7MB)，SHA256 与可信目录完全一致。
- **质量基线**：48 测试文件 / 271 用例全部通过，语句覆盖率 71.76%；typecheck 全绿；CI (GitHub Actions) 含质量门禁 + E2E + Windows 打包。
- **完整演示闭环**：自然语言 → 路由 → 分析 → 固定 commit → 双审批 → 迅雷下载 → 验证 → 交接包 → Agent B，已按 deepseek-harness 场景实测走通。

5.2 待补齐 (风险清单)

项目	说明	优先级
Windows 11 实机验收	当前在开发机验证，缺真实 Win11 + 签名证书	高
受控安装	官方安装器静默安装 (白名单参数)，补普通用户闭环	中
可信目录扩展	目前 12 个资源，需覆盖更多生态 (winget/choco/conda 等)	中
E2E 覆盖	当前 1 个 spec，需补主路径	低

5.3 技术栈与可维护性

Electron 43 + React 18 + TypeScript + Vite + sql.js + Vitest/Playwright，类型全绿、文档 17 份、演示脚本齐备——具备移交与继续开发的条件。

6. 商业化思考

6.1 商业模式选项

模式	形态	依据
B2C：会员增值	迅雷会员中"AI 资源准备"能力包：下载加速 + 供应链验证 + 一键工作区	复用迅雷会员体系与用户基数
B2B：企业供给层	企业内部部署：AI Agent 的依赖/环境供给 + 审计合规（金融/政企场景强需求）	治理层价值只有企业买单
开放平台：供应链即服务	把可信目录 + 验证链开放为 API/SDK，任何 Coding Agent 可接入	类比 CDN / Docker Hub 的"基础设施"位
交易分成	软件/模型/数据分发的可信交付通道，按量分成	迅雷分发网络的自然延伸

6.2 竞争定位

不与 Claude Code / Codex 竞争写代码，做复杂任务，做它们的供给方与兜底层。

差异化护城河：P2SP 物理网络（20 年积累，Agent 公司无法复制）+ 供应链验证 + 企业级审计链。

目前Coding Agent 正在企业化落地，供应链安全与治理是 2025-2026 最热的 Agent 议题，此时入场供给层是窗口期。

6.3 收入测算逻辑（示意）

- B2C：以会员 ARPU 提升衡量，不单独计费，降低获客成本；
- B2B：按席位/年授权 + 私有目录定制，客单价高、续约依赖审计价值；
- 平台：按下载量/验证次数/分发抽成，规模效应随 Agent 生态增长。

6.4 风险与对策

风险	对策
供应链责任（分发被投毒）	可信目录版本化 + 双校验 + 可追溯，责任边界清晰
巨头下场（GitHub/Anthropic 自建分发）	迅雷网络是物理资产，短期无法复制；开放生态绑定
普通用户需求不足	受控安装补闭环；核心短期打 B2B 与开发者
政策/合规	分发内容审核 + 区域合规，按平台标准建设

6.5 三步走路线图

1. **P0 供给层（当前）**：可信下载 + 验证 + 交接，打通讯雷 P2SP，完成开发者场景闭环。
2. **P1 闭环层**：受控静默安装（白名单安装器）+ 可信目录生态扩展，覆盖普通用户。
3. **P2 平台层**：开放供应链 API，接入第三方 Agent 生态，形成“供给即服务”商业模式。

7. 一句话总结

XL-Agent 不解决“AI 能不能干活”，解决“AI 干活时每一步是否可信、可控、可审计、可复现”。能力层认输，治理层碾压；Coding Agent 负责“聪明地干活”，迅雷负责“可靠地供给”——聪明会越来越便宜，供给的信任不会。

源文档：product-report-2026-08.md · 生成日期：2026-08-15